

SIMATS ENGINEERING ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence

COURSE CODE:CSA17

COURSE OUTCOME COVERED

CO2: Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)

Assessment Tool Weightage: 50%

QUESTIONS: 3

Duration: 1 Hour
Total Marks:50

Annexure A

Scenario Based Assignment

Code of Conduct:

I M/ 'Arghim' 192404224 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: M. Arghim

20+20+23+18+13
=94
HP

Scenario Based Assignment

Student ID: MARSINI | Register Number: 192424322

1. Scenario

A fintech startup is developing an AI agent to compete in a simulated auction bidding game against other automated bidders. The AI must compete against adversarial bidding agents, make decisions within strict time limits per bidding round, evaluate complex market states with multiple possible bids, and the decision tree of possible bid sequences is extremely large, making computation expensive.

Tasks:

- Explain how the Minimax algorithm models this problem.
- Analyze the computational challenges of large game/decision trees.
- Explain how Alpha-Beta pruning improves efficiency with detailed reasoning.
- Design an evaluation function and justify the factors considered.
- Recommend a strategy for balancing optimality and real-time performance.

2. Scenario

A marine research team deploys an autonomous underwater vehicle (AUV) to locate survivors near a sunken vessel. The environment is highly dynamic - underwater currents shift debris and visibility, and travel cost changes with depth and current strength. The AUV has access to: a heuristic estimate (straight-line distance to the last known survivor location), partial real-time sonar updates about newly obstructed paths, and variable movement costs depending on current strength and depth. Given the urgency of survivor rescue, the AUV must quickly decide routes with minimum delay and risk.

Tasks:

- Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty.
- Explain how A* would handle the same situation, including how it balances cost and heuristic.
- Analyze the impact of heuristic accuracy on both algorithms.
- Discuss how dynamic changes (shifting debris and visibility) affect search performance.
- Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety.

3. Scenario

A cloud computing company implements an AI system to balance workloads across thousands of servers in its data centers. The objective is to minimize energy consumption, response latency, and hardware wear. The system must continuously redistribute tasks based on real-time server load. However, the search space is extremely large, incoming request

patterns change dynamically, and the system may get stuck in locally optimal load distributions.

Tasks:

Formulate this as an optimization problem and explain why traditional search is inefficient.

- i. Explain how Hill Climbing would work in this scenario and identify its limitations.
- ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation.
- iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations.
- iv. Recommend the best approach and justify your choice based on scalability and adaptability.

AI Agent for simulated Auction Bidding

A fintech startup is developing an AI Agent that competes against other automated bidders. Since each bidder tries to maximize its own benefit, since each the problem can be treated as an adversarial search problem. Minimax and Alpha-Beta pruning are suitable techniques.

(i) MINIMAX Algorithm

MINIMAX Algorithm is used when two players have opposite objectives

- * MAX represents our AI Agent.
- * MIN represents the opponent
- * Each node represents a possible bidding decision
- * MAX chooses the highest valued outcome
- * MIN chooses the lowest valued outcome

Thus, minimax helps the AI select the best quality bidding strategy assuming opponent also plays optimally.

2) Computational challenges of large game tree

If there are b possible actions at each level and search depth is d , number of nodes $\approx O(b^d)$

1) number of possible game states becomes extremely large

Searching the complete tree takes more time.

memory requirements can increase significantly.

Real time bidding decisions becomes difficult

Evaluating every possible future sequence is computationally expensive.

iii) How Alpha - Beta pruning improves efficiency

Alpha - Beta pruning removes branches that cannot affect the final decisions.

It maintains two values:-

* Alpha (α): best value currently generated for max

* Beta (β): - best value currently guaranteed for min

when $\alpha \geq \beta$, remaining branches can be safely ignored because they cannot improve the final decision.

Advantages:-

* Reduces the number of nodes evaluated.

* produces the same optimal decisions as min max

* Allows deeper search within the same time

* limit

* Improves real time decision making

* Reduces the computational cost.

with good move ordering, Alpha Beta pruning can

reduce complexity from $O(b^d)$ to $O(b^{d/2})$ best case.

(i) How Greedy Best-First search would behave in this

* Greedy Best-First search (GBFS) always expands the node that appears to be closest to the goal according to the heuristic.

* It uses only the heuristic's value $h(n)$ and ignores the actual cost.

* In this AUV scenario, it will always choose the location that seems nearest to the last known survivor position.

* strength:- fast, memory efficient and works well when heuristic is accurate.

* weakness:- can get trapped in wrong paths, if heuristic is misleading due to dynamic environment and uncertainty.

(ii) How A^* would handle the same situations and how it balances heuristic

* A^* uses both the actual cost from start $g(n)$ and heuristic $h(n)$ to the goal.

* It evaluates nodes using $f(n) = g(n) + h(n)$

* It balances b/w the shortest known path and the distance to the goal.

Thus, A^* is the optimal and more reliable in uncertain environments because it considers both depth-dependent cost and heuristic estimate.

(ii) Impact of heuristic accuracy on A^* algorithms.

- * If heuristic is accurate (close to actual cost):

Both GBFS and A^* perform well

- * If heuristic is inaccurate:

GBFS may choose wrong paths and

- * A^* still finds optimal paths but may explore more nodes

(iii) How dynamic changes affect search performance.

- * In underwater environments, current: visibility and debris may change.

- * These changes can make the heuristic outdated.

- * GBFS is more affected because it also relies only

- * only one heuristic

- * A^* can adapt because it also considers path cost.

i) optimization problems and why traditional search is inefficient

* This is a large scale optimization problem where the objective is to minimize energy consumption, response latency and hardware while distributing workloads

* The search space is extremely large because incoming requests are continuous, patterns change dynamically and there are many possible distributions.

Traditional Search (like BFS, DFS or even A^*) is inefficient because:

- 1) The state space is enormous
- 2) It requires exploring a huge number of combinations
- 3) It cannot adapt quickly to real-time changes

ii) How Hill climbing works and its limitations

Hill climbing starts with an initial solution and moves to a neighbouring solution that improves the objective reduces energy the latency.

It continues until no better neighbour solution.

Limitations:

- 1) can get stuck in minimal local maxima
- 2) cannot escape plateaus where all neighbours have same value.
- 3) not good for large complex search space.
- 4) no guarantee of finding the global optimum.

iii) How simulated Annealing or Genetic Algorithm overcome limitations

simulated Annealing (SA):-

- * Allows accepting worse solutions with a probability that decreases over time.

- * This helps escape local optima and explore more stuck.

Genetic Algorithm

- * maintains a population of solutions.
- * uses selection, crossover and mutation to explore search space.
- * can handle large, complex and dynamic environments efficiently.

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding, misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues, some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

$$20 + 20 + 23 + 18 + 13$$

$$= 94$$

10